

Programovací jazyky a překladače

Přednáška 5, poznámky

Jan Voráček

VŠPJ Jihlava, voracek@vspj.cz

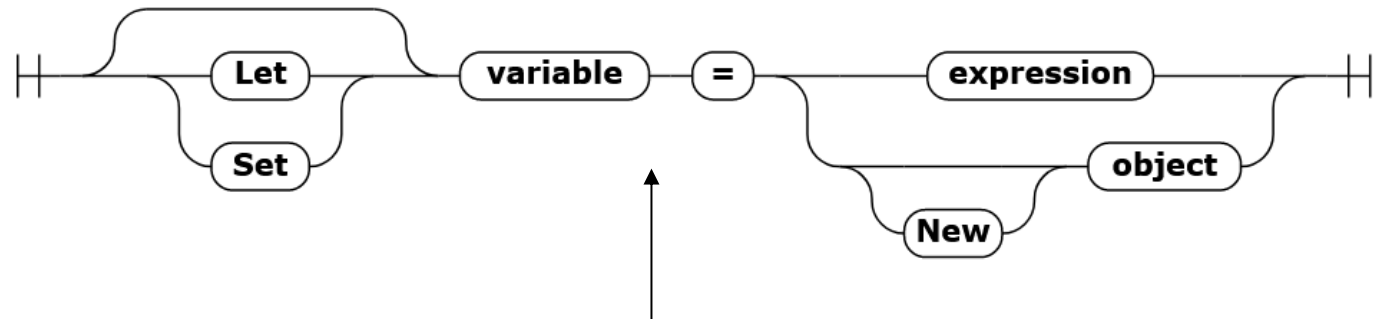
Témata

- Administrativa: prezentace dnes nejsou
 - V Moodle jsou dostupné informace k semestrálnímu projektu
- Semestrální projekt
- Vzorové příklady ke gramatikám
- Konečné automaty
 - Princip
 - Reprezentace a návrh
 - Minimalizace
- Vzorové příklady ke konečným automatům (bude-li čas)

Opakovací příklady ke gramatikám, shrnutí



- Jeden jazyk může být generován více gramatikami
- Při konstrukci překladačů nás zajímají:
 - Regulární gramatiky
 - Bezkontextové gramatiky
- Jazyky charakterizujeme:
 - Výčtem
 - Slovně
 - Množinově
 - Formalizovaným zápisem (RE, Backusova–Naurova forma[BNF])
 - Gramatikou
 - Automatem



Reprezentace jazyků slovně a množinově

- Jazyk, tvořený všemi řetězci, obsahujícími libovolný počet znaků x .

$$\{x^n \mid n > 0\}$$

- Jazyk, tvořený všemi řetězci, obsahujícími nejméně jeden znak x , následovaný vždy stejným počtem znaků y

$$\{x^n y^n \mid n > 0\}$$

- Jazyk, tvořený všemi řetězci, obsahujícími nejméně jeden znak x , následovaný vždy nejméně jedním řetězcem y , přičemž počet znaků x a y nemusí být shodný

$$\{x^n y^m \mid n, m > 0\}$$

Reprezentace jazyků regulárními výrazy (RE)

- Popište předchozí jazyk pomocí regulárního výrazu (RE)

x^+y^+

nebo

xx^*yy^*

- Napište RE generující žádný nebo více znaků x nebo žádný nebo více znaků y

$x^* \mid y^*$

- Napište RE generující žádný nebo více znaků a nebo b

$(a \mid b)^*$

- Napište RE generující jeden znak a a žádný nebo více znaků b

$a \mid b^*$

Další příklady na jazyky a gramatiky

- Napište několik řetězců, generovaných RE $(aab/ab)^*$
 $empty(\varepsilon), \quad ababab, \quad aababaab$
- Je RE $(abc)^+$ ekvivalentní výrazu $(abc)(abc)^*$?
 ANO
- Vyjádřete vzory pro identifikátory a čísla s pevnou desetinnou tečkou jako RE
 $L(L \mid D)^* \quad (D^*D.D^*) \mid (D^*.DD^*)$
- Lze definovat jazyk, párující oddělovače, pomocí RE?
Protože tato konstrukce používá čítač, nelze k její definici použít RE, ale nejméně CFG, například s těmito pravidly:

$$S \rightarrow xSy \mid xy$$

Jazyk generovaný regulárním výrazem (RE)

Který z následujících jazyků nad slovníkem $\{0,1\}$ je reprezentován tímto regulárním výrazem:

$$(0+1)^*0(0+1)^*0(0+1)^*$$

Odpověď: je to množina všech řetězců:

- a) Obsahujících podřetězec 00
- b) Obsahujících nejvýše dvě nuly
- c) Obsahujících nejméně dvě nuly
- d) Začínajících nebo končících buď 0 nebo 1

Jazyk generovaný regulární gramatikou (RG)

Který z jazyků a) – d) je generován následující gramatikou:

$$S \rightarrow aS \mid bS \mid \varepsilon$$

Odpověď:

a) $\{a^n b^m \mid n, m \geq 0\}$

b) $\{w \in \{a,b\}^* \mid w \text{ má stejný počet znaků } a \text{ i } b\}$

c) $\{a^n \mid n \geq 0\} \cup \{b^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 0\}$

d) $\{a,b\}^*$

Slovně charakterizujte následující RE

$0(0|1)^*0$

Řetězce začínající a končící 0

$((\varepsilon|0)1^*)^*$

Libovolné řetězce

$(0|1)^*0(0|1)(0|1)$

Řetězce, obsahující 0 na třetí pozici zprava

$0^*10^*10^*10^*$

Řetězce, obsahující právě tři jedničky

Popis jazyka pomocí RE a gramatiky

Jazyk všech řetězců nad abecedou $\{0,1\}$ takový, že každá 0 je okamžitě následována nejméně jednou 1. Nesmí být prázdný.

- RE: $1(011^*)^*$
- Bezkontextová (CFG) gramatika 1

$$S \rightarrow SS|0A|1$$

$$A \rightarrow 1A|1$$

RE a gramatika pro totožný jazyk s možným prázdným vstupem

- RE: $1^*(011^*)^*$
- Bezkontextová gramatika 2

$$S \rightarrow AB$$

$$A \rightarrow \varepsilon \mid 1A$$

$$B \rightarrow \varepsilon \mid CB$$

$$C \rightarrow 01A$$

Popis jazyka pomocí gramatiky

- Jazyk všech párových oddělovačů typu (), [] nebo {}.

$$S \rightarrow (S)S \mid [S]S \mid \{S\}S \mid \varepsilon$$

- Prázdný jazyk všech řetězců, tvořených 0 a 1 takový, že počet 0 odpovídá počtu 1

$$S \rightarrow 0S1S \mid 1S0S \mid \varepsilon$$

- Prázdný jazyk všech řetězců nad abecedou {0,1} takový, že se v něm nemůže vyskytnout podřetězec 011

$$S \rightarrow AB$$

$$A \rightarrow 1A \mid \varepsilon$$

$$B \rightarrow 01B \mid 0B \mid \varepsilon$$

Konečný automat (KA), definice



Konečný automat

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Q : (konečná) množina stavů
- Σ : vstupní abeceda
- δ : přechodová funkce
 $\delta_{NKA} : Q \times \Sigma \rightarrow 2^Q$ $\delta_{DKA} : Q \times \Sigma \rightarrow Q$
- q_0 : počáteční stav
- F : množina koncových stavů

Konečný automat, vzorový příklad

Mějme automat $M = (Q, T, d, q_0, F)$, kde:

$Q = \{q_0, q_1\}$

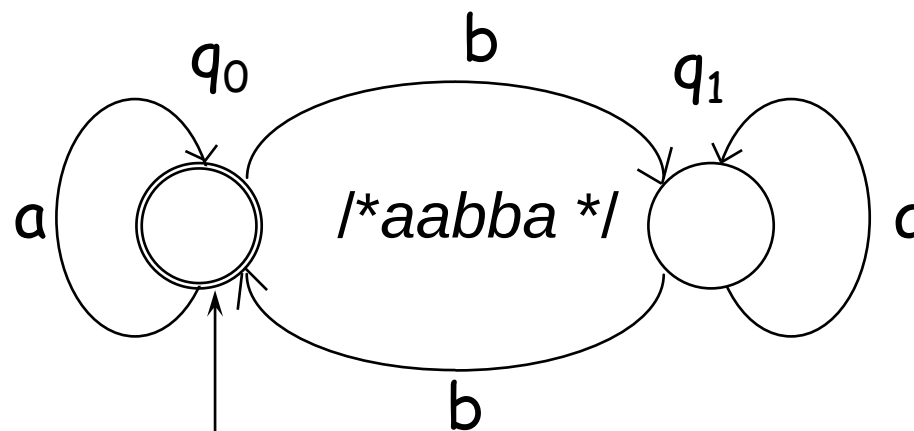
$T = \{a, b\}$

$F = q_0$

d je dáno tabulkou

q	s	$\delta(q,s)$			
q_0	a	q_0	δ	a	b
q_0	b	q_1	q_0	q_0	q_1
q_1	a	q_1	q_1	q_1	q_0
q_1	b	q_0			

Automat lze popsat také stavovým (přechodovým) diagramem:



Metodika návrhu KA: $NKA \rightarrow DKA \rightarrow MDKA$

- 1) Redundantní automatický nebo „manuální“ návrh (např. na základě definice), vedoucí na nedeterministický KA (NKA)
- 2) Konverze NKA na deterministický KA (DKA)
- 3) Minimalizace DKA (MDKA)

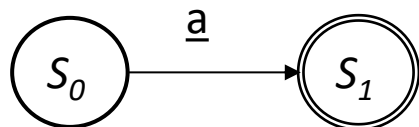
The screenshot shows a web application for converting regular expressions to minimum DFA. The interface includes a header with navigation links (Toolbox, Home, Repository, Issues) and GitHub repository statistics. The main content area is divided into three sections: Introduction, Supported grammars, and Examples. The input field contains the regular expression `(a|b)*abb(a|b)*`. Below the input, there is a table showing the DFA states and their transitions.

DFA STATE	Min-DFA STATE	TYPE	a	a,b	b
{A,C}	1		2		1
{B}	2		2		3
{D}	3		2		4
{E,F,G,H,I}	4	accept		4	

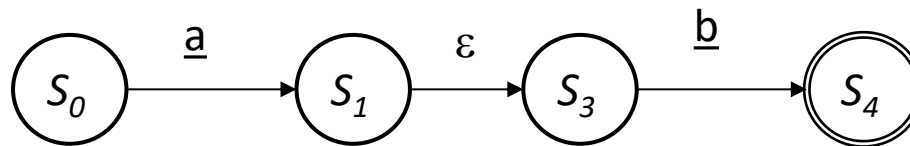
Below the table, a state transition diagram is shown with four states (1, 2, 3, 4). State 1 is the start state, and state 4 is the accept state. Transitions are labeled with 'a' and 'b'.

Zdroj: https://cyberzhg.github.io/toolbox/min_dfa?regex=KGF8Yikq

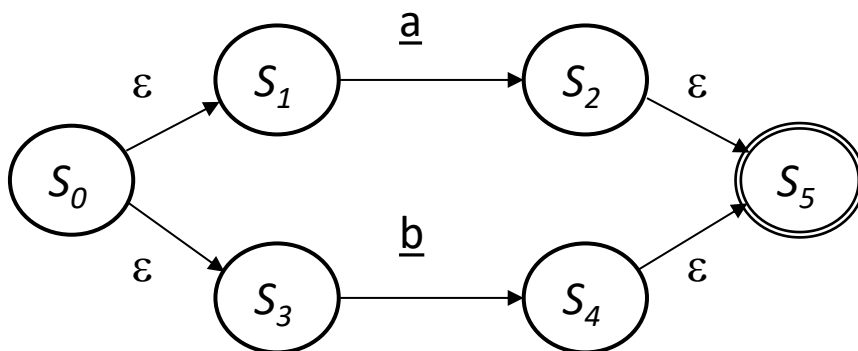
Thompsonova metoda návrhu NFA



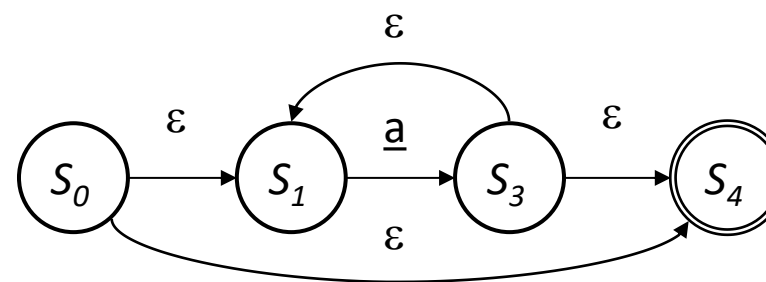
NFA pro $\underline{a}$



NFA pro $\underline{ab}$



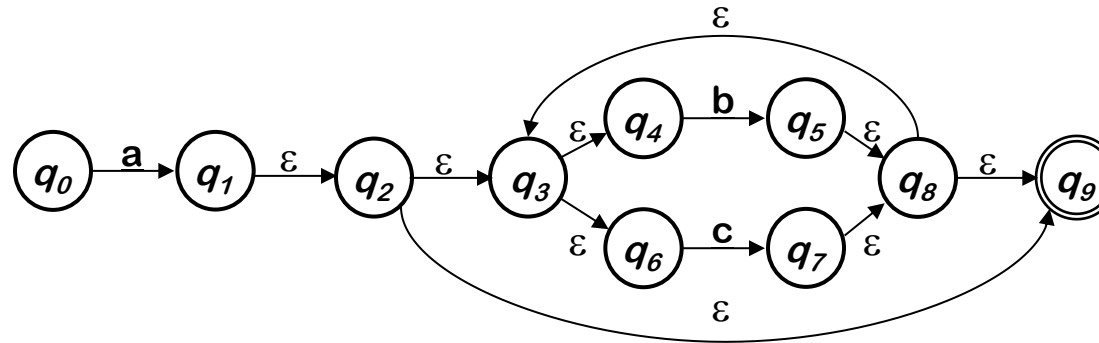
NFA pro $\underline{a} \mid \underline{b}$



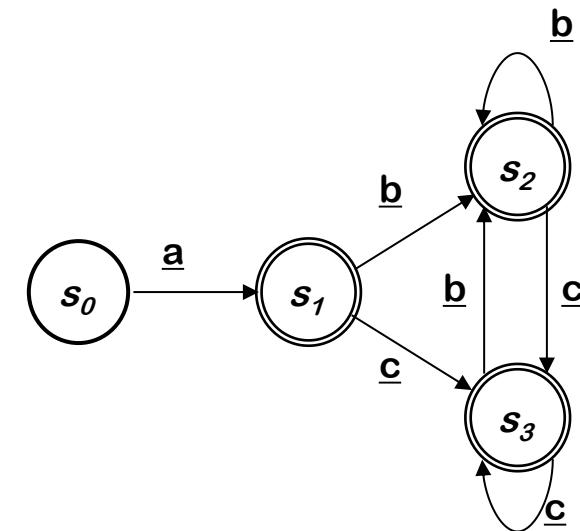
NFA pro $\underline{a}^*$

Konstrukce NFA a konverze na DFA

$a(b \mid c)^*$



		ϵ -closure(move($s, *$))		
	NFA states	<u>a</u>	<u>b</u>	<u>c</u>
s_0	q_0	$q_1, q_2, q_3, q_4, q_6, q_9$	<i>none</i>	<i>none</i>
s_1	$q_1, q_2, q_3, q_4, q_6, q_9$	<i>none</i>	$q_5, q_8, q_9, q_3, q_4, q_6$	$q_7, q_8, q_9, q_3, q_4, q_6$
s_2	$q_5, q_8, q_9, q_3, q_4, q_6$	<i>none</i>	s_2	s_3
s_3	$q_7, q_8, q_9, q_3, q_4, q_6$	<i>none</i>	s_2	s_3



Metodika návrhu KA: $NKA \rightarrow DKA \rightarrow MDKA$

- 1) Redundantní automatický nebo „manuální“ návrh (např. na základě definice), vedoucí na nedeterministický KA (NKA)
- 2) Konverze NKA na deterministický KA (DKA)
- 3) Minimalizace DKA (MDKA)

The screenshot shows a web application for converting regular expressions to minimum DFA. The interface includes a header with navigation links (Toolbox, Home, Repository, Issues) and GitHub repository statistics. The main content area has three sections: Introduction, Supported grammars, and Examples. The input field contains the regular expression `(a|b)*abb(a|b)*`. A **CONVERT** button is visible. Below the input, the DFA is displayed as a table and a diagram.

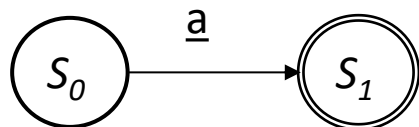
DFA: <https://cyberzhg.github.io/toolbox/nfa2dfa?regex=KGF8YikqYVJlKGF8Yikq>

DFA STATE	Min-DFA STATE	TYPE	a	a,b	b
{A,C}	1		2		1
{B}	2		2		3
{D}	3		2		4
{E,F,G,H,I}	4	accept		4	

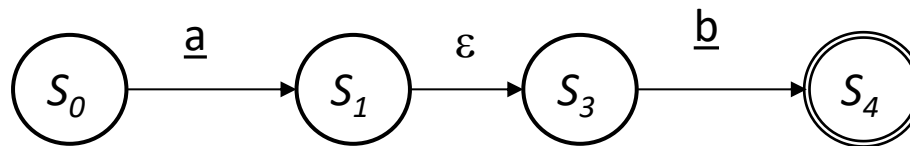
The DFA diagram shows four states: 1 (start), 2, 3, and 4 (accept). Transitions are: 1 to 2 on 'a', 2 to 1 on 'b', 2 to 3 on 'a', 3 to 2 on 'b', 3 to 4 on 'a', and 4 to 3 on 'b'.

Zdroj: https://cyberzhg.github.io/toolbox/min_dfa?regex=KGF8Yikq

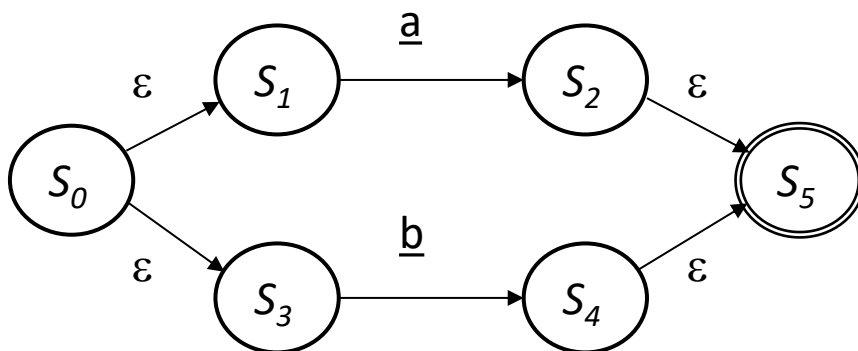
Thompsonova metoda návrhu NFA



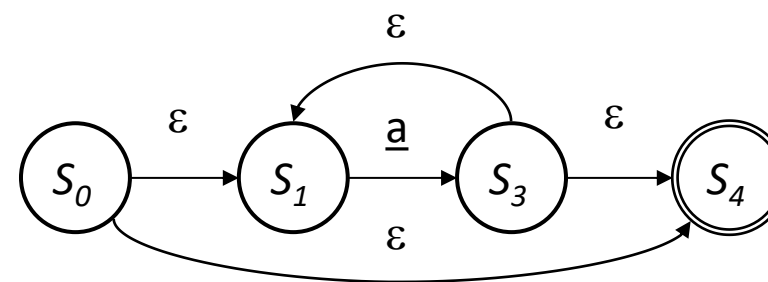
NFA pro $\underline{a}$



NFA pro $\underline{ab}$



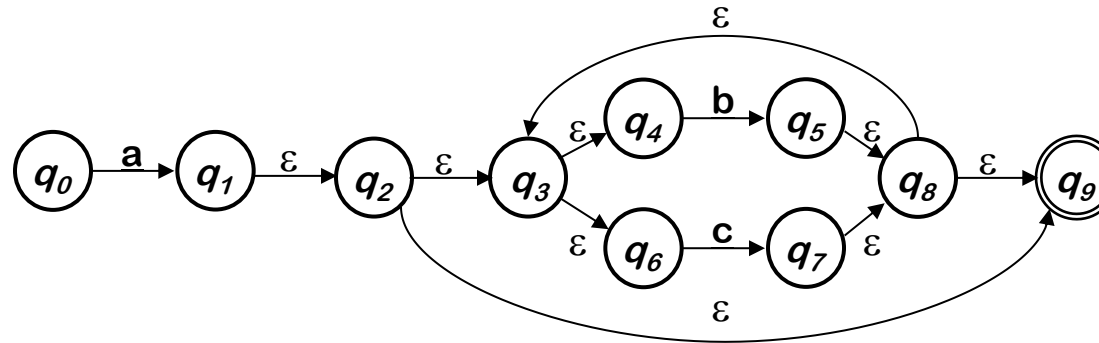
NFA pro $\underline{a} \mid \underline{b}$



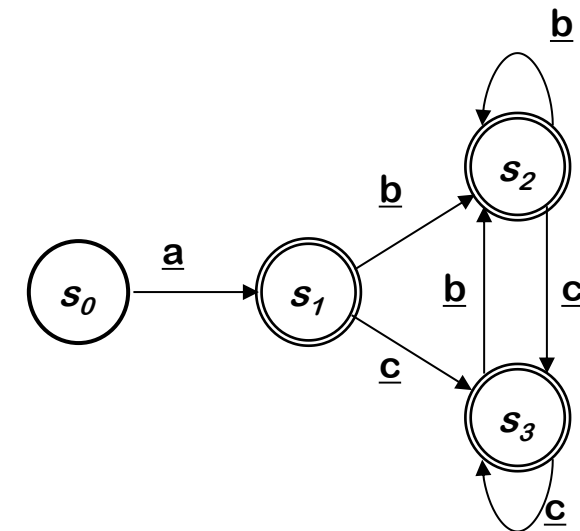
NFA pro $\underline{a}^*$

Konstrukce NFA a konverze na DFA

$a(b \mid c)^*$



		ϵ -closure(move($s, *$))		
	NFA states	<u>a</u>	<u>b</u>	<u>c</u>
s_0	q_0	$q_1, q_2, q_3, q_4, q_6, q_9$	<i>none</i>	<i>none</i>
s_1	$q_1, q_2, q_3, q_4, q_6, q_9$	<i>none</i>	$q_5, q_8, q_9, q_3, q_4, q_6$	$q_7, q_8, q_9, q_3, q_4, q_6$
s_2	$q_5, q_8, q_9, q_3, q_4, q_6$	<i>none</i>	s_2	s_3
s_3	$q_7, q_8, q_9, q_3, q_4, q_6$	<i>none</i>	s_2	s_3



Minimalizace KA, princip

Klíčová slovo při redukci skupiny podmíněně redundantních stavů je

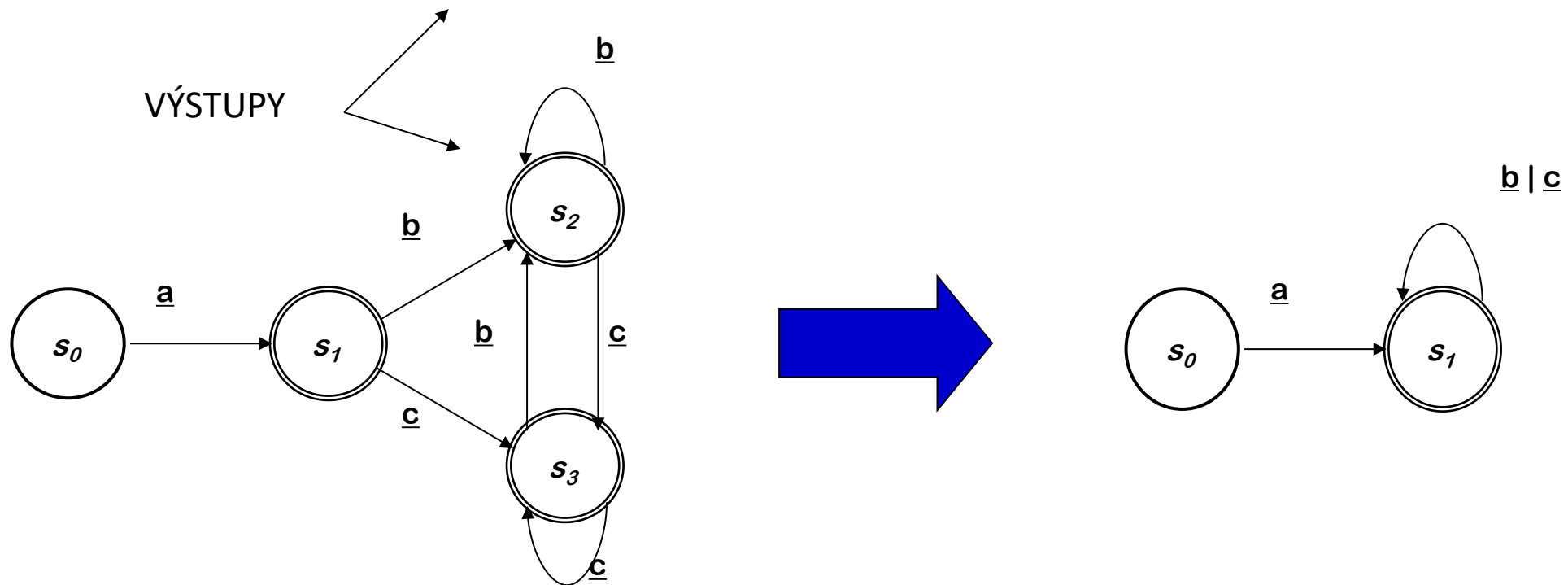
NEROZLIŠITELNOST.

Znamená to, že je-li přesun z více stavů na stejné vstupní symboly shodný, tj. vedoucí do stejné skupiny stavů, nelze tyto stavy rozlišit a dochází k jejich sloučení a následné další analýze. Ta může buď novou skupinu zachovat, nebo rozdělit na další podskupiny. Tímto způsobem může dojít ke snížení počtu původních stavů.

Blíže viz následující příklad.

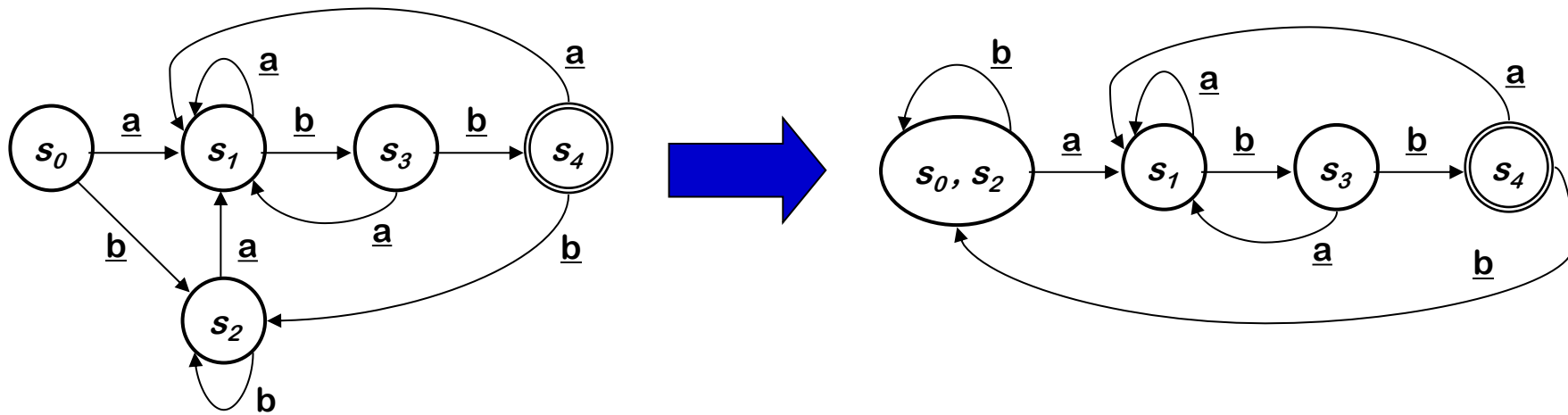
Minimalizace DFA

	<i>Aktuální dělení</i>	<i>Rozdělení při:</i>		
		<u>a</u>	<u>b</u>	<u>c</u>
P_0	$\{s_1, s_2, s_3\} \{s_0\}$	<i>ne</i>	<i>ne</i>	<i>ne</i>



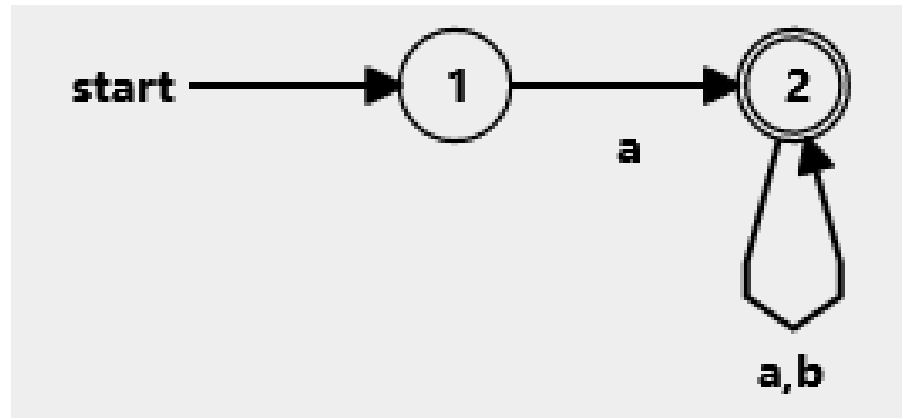
Minimalizace složitějšího KA pro RE $(a|b)^*abb$

	<i>Aktuální dělení</i>	<i>Rozdělení při a</i>	<i>Rozdělení při b</i>
P_0	$\{s_4\} \{s_0, s_1, s_2, s_3\}$	ne	$\{s_0, s_1, s_2\} \{s_3\}$
P_1	$\{s_4\} \{s_3\} \{s_0, s_1, s_2\}$	ne	$\{s_0, s_2\} \{s_1\}$
P_2	$\{s_4\} \{s_3\} \{s_1\} \{s_0, s_2\}$	ne	ne

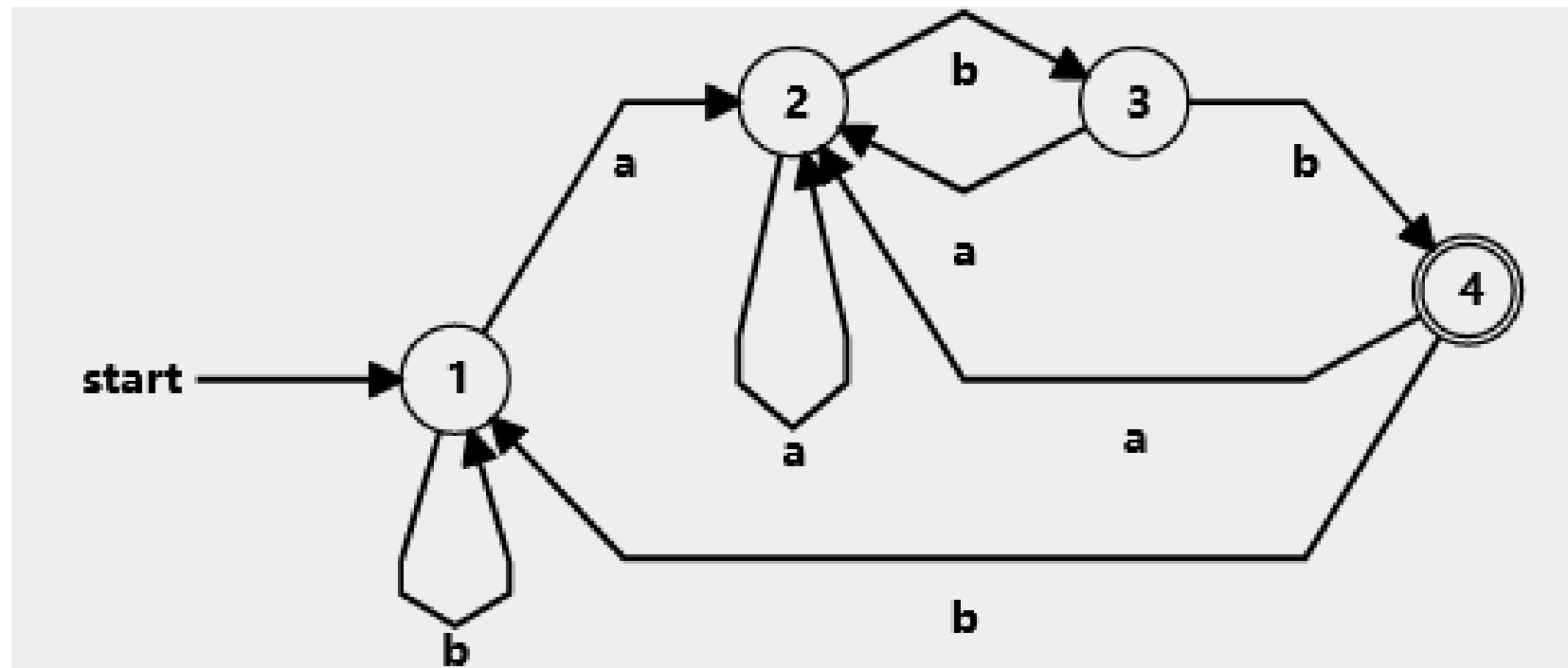


Řešení od CyberZHG

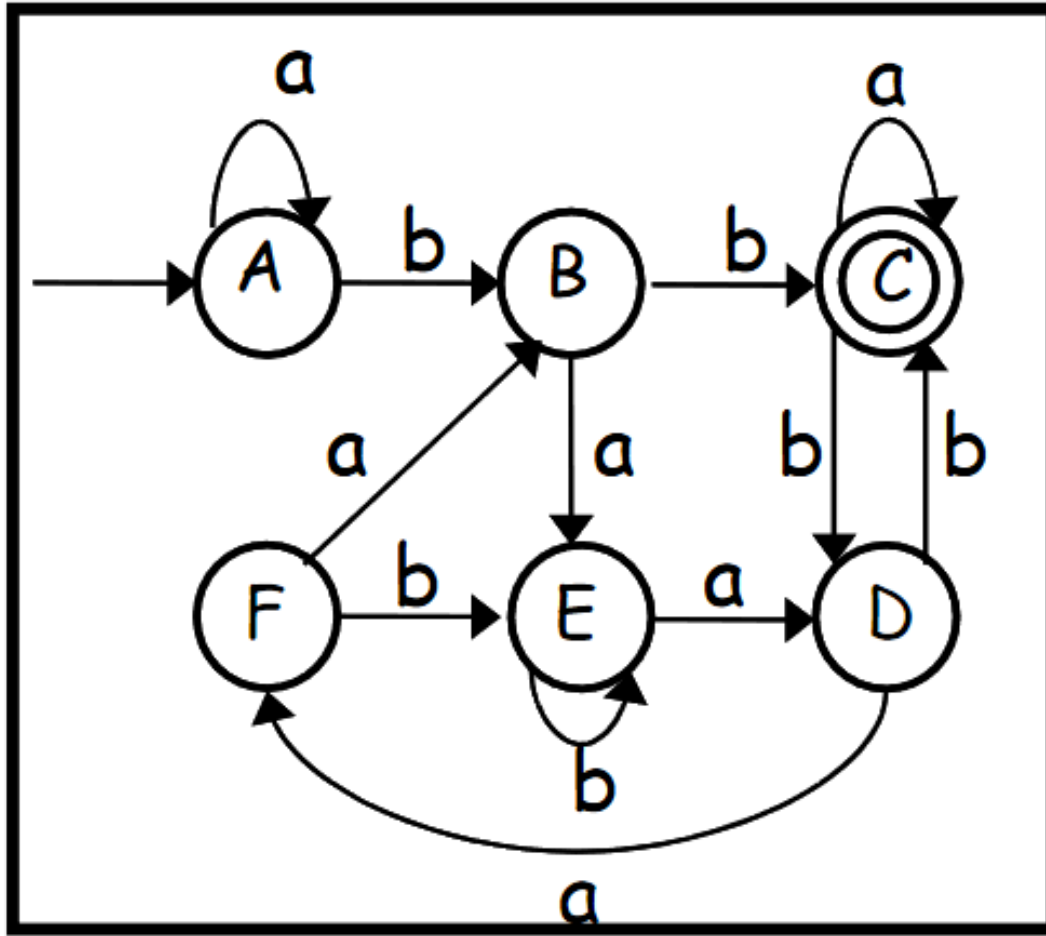
$a(b|c)^*$



$(a|b)^*abb$



Minimalizace KA 6 → 4



1

	a	b
A	A	B
B	E	C
C	C	D
D	F	C
E	D	E
F	B	E

	a	b
A	I	I
B	I	II
D	I	II
E	I	I
F	I	I

2

	a	b
A	i	ii
E	ii	i
F	ii	i

3

	a	b
B	i	ii
D	i	ii

4

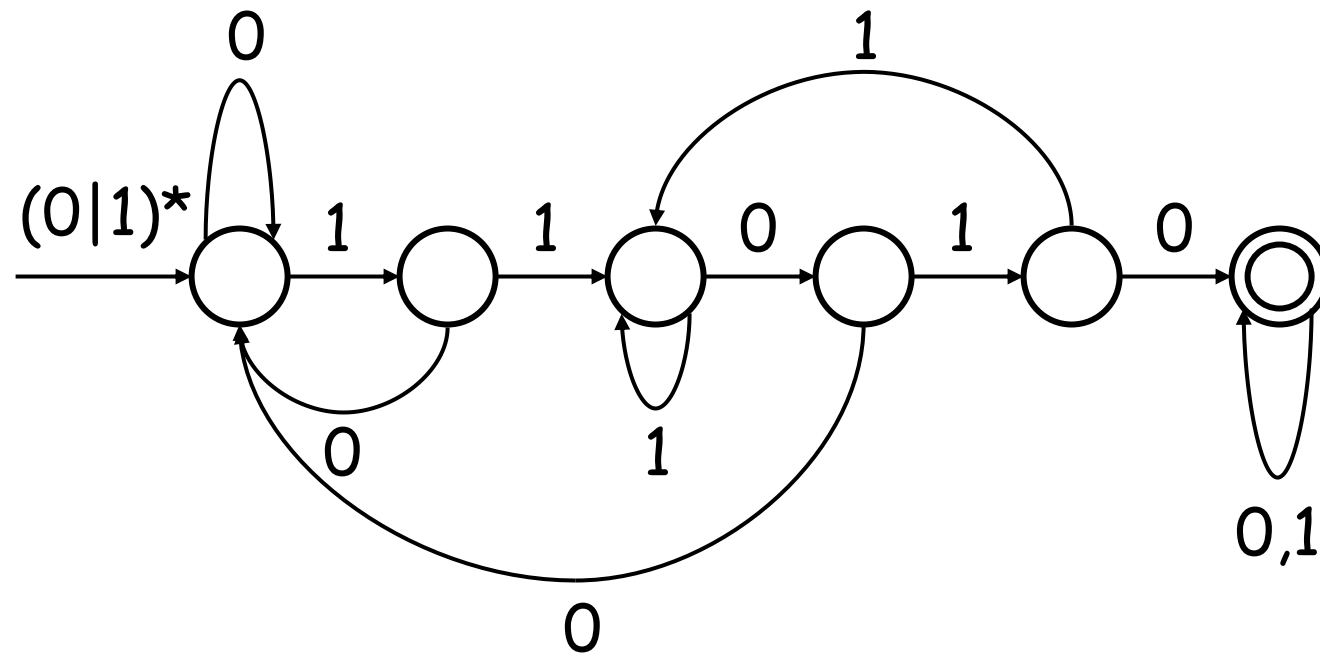
	a	b
E	aa	a
F	aa	a

Konečný automat, příklad 1

Navrhněte konečný automat nad abecedou $A=\{0,1\}$, přijímající následující jazyk:

A^* obsahující podřetězec 11010

Řešení:

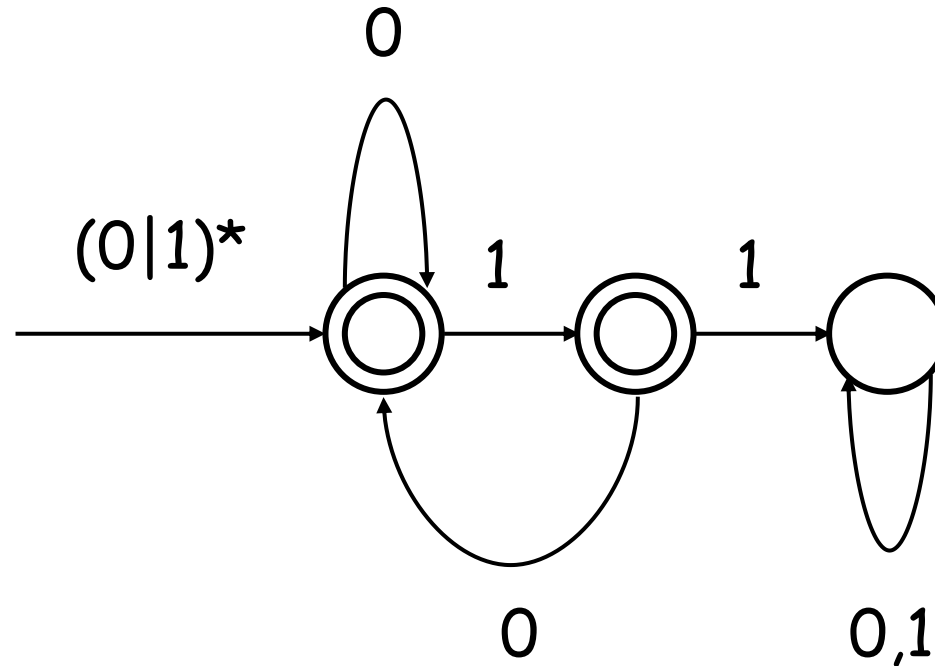


Konečný automat, příklad 2

Navrhněte konečný automat nad abecedou $A=\{0,1\}$, přijímající následující jazyk:

A^* kromě slov, obsahujících podřetězec 11

Řešení:



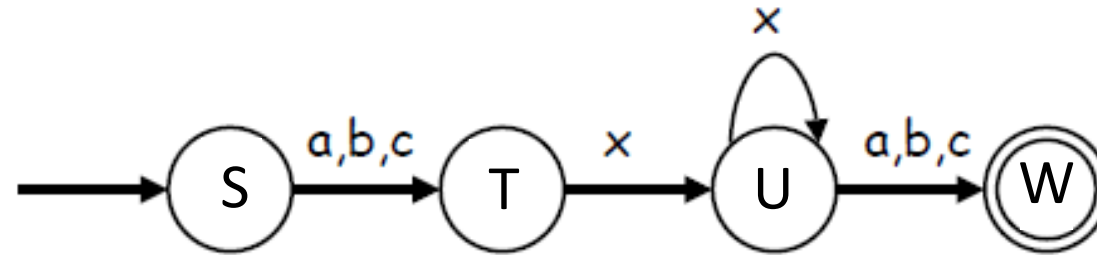
Vztah mezi RE, RG a FA

Pro následující regulární výraz (RE):

$(a|b|c)x^+(a|b|c)$

navrhněte odpovídající konečný automat (FA) a regulární gramatiku (RG).

- **FA:**



- **RG:**

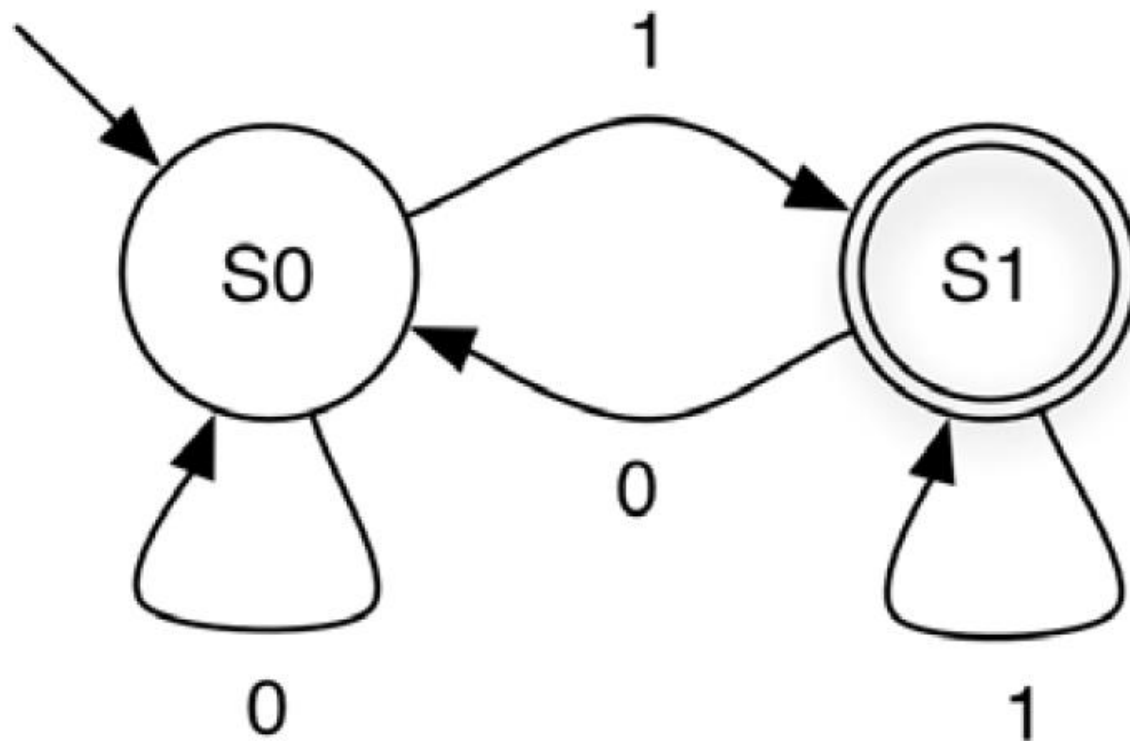
$S \rightarrow aT \mid bT \mid cT$

$T \rightarrow xU$

$U \rightarrow xU \mid aW \mid bW \mid cW$

$W \rightarrow \text{acc.}$

Jaký jazyk přijímá tento KA?



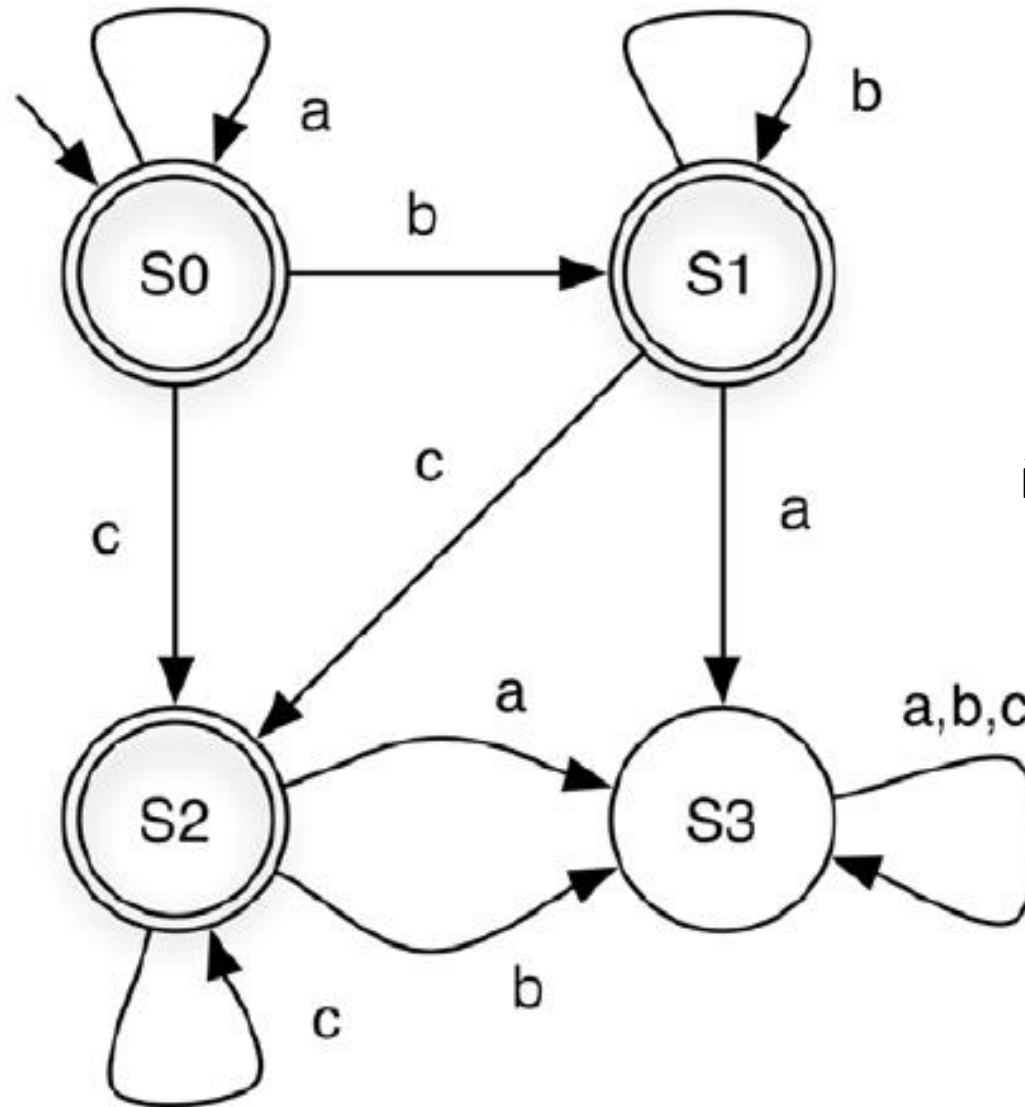
Řešení vyjádřete slovně a zapište pomocí regulárního výrazu.

Řešení předchozího příkladu

Řetězce 0 a 1, končící 1

$$(0|1)^*1$$

Jaký jazyk přijímá tento KA?

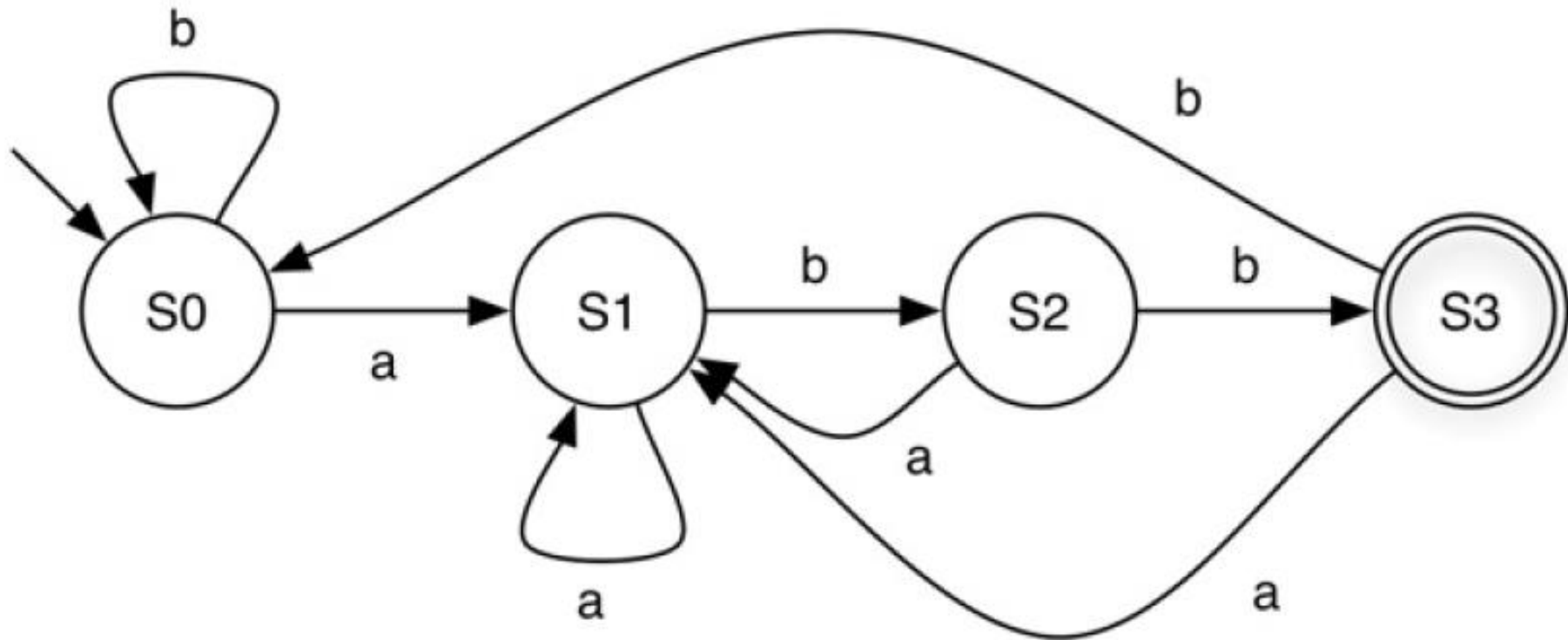


Řešení запиšte pomocí regulárního výrazu (RV).

Řešení předchozího příkladu

$a*b*c*$

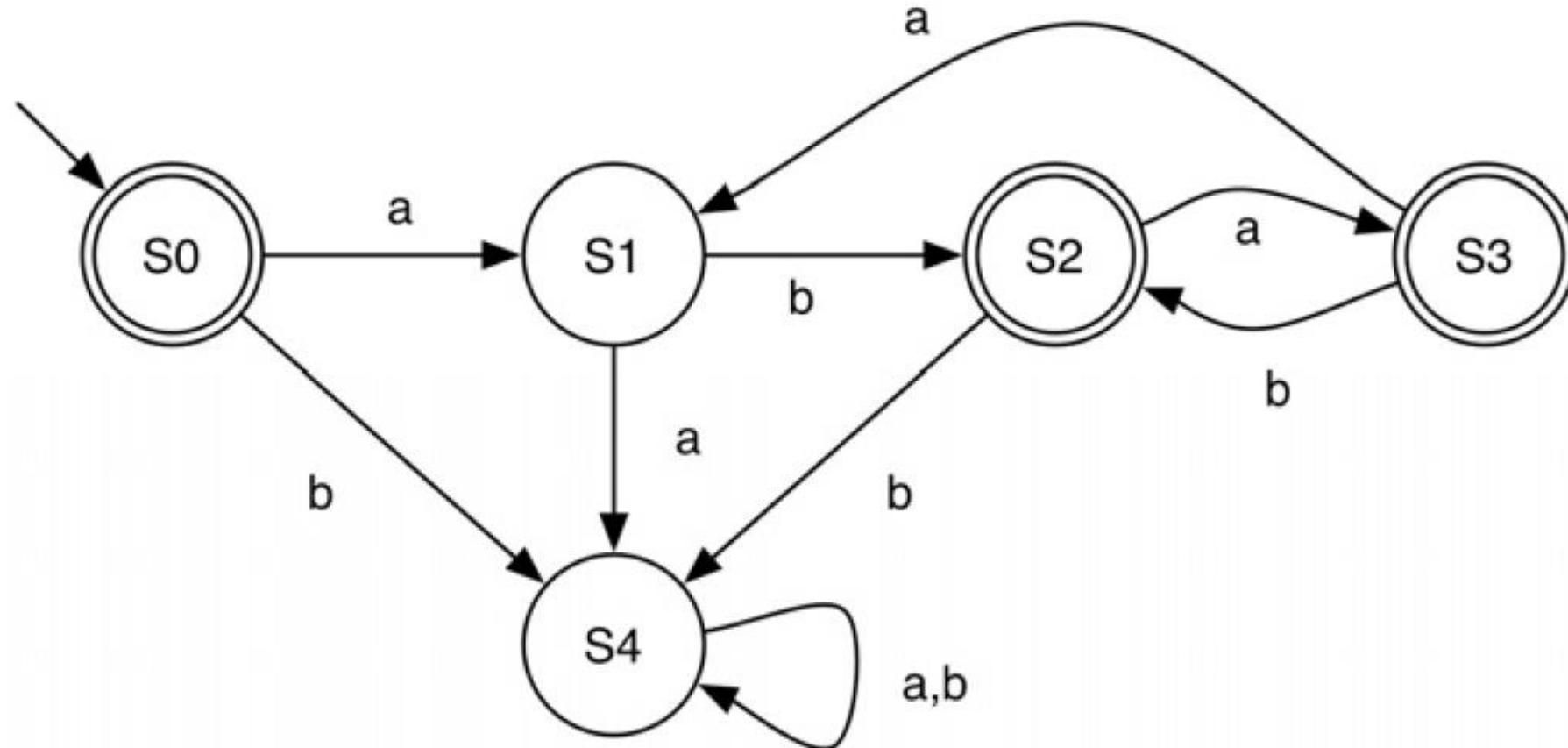
Jaký jazyk, vyjádřeno RV, přijímá tento KA?



Řešení předchozího příkladu

$(a | b)^*abb$

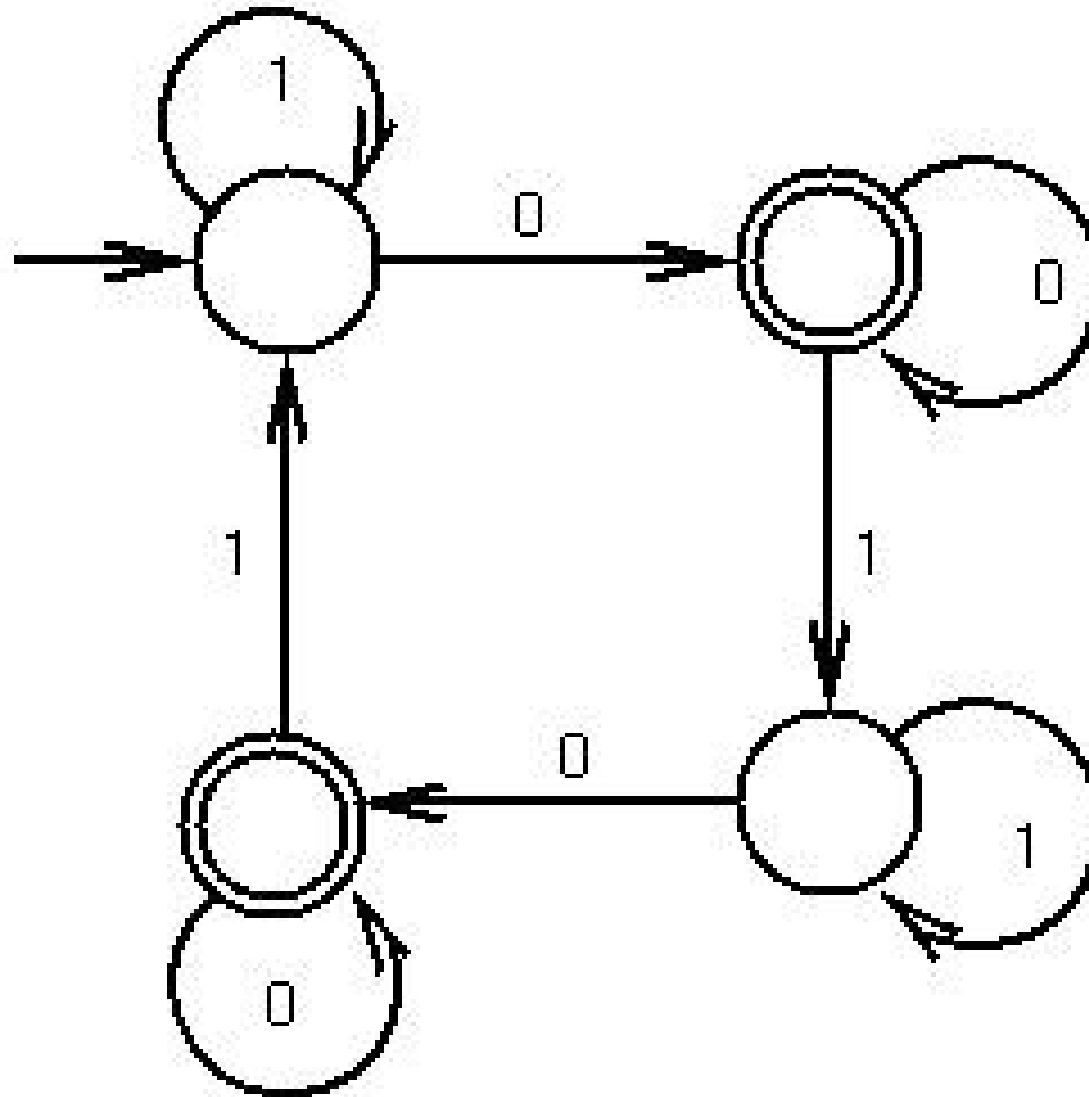
Jaký jazyk, vyjádřeno RV, přijímá tento KA?



Řešení předchozího příkladu

$(ab|aba)^*$

Jaký jazyk, vyjádřeno RV, přijímá tento KA?



Řešení předchozího příkladu

$$(1^* (00^* 11^* 00^* 11^*)^* (00^* \mid 00^* 11^* 00^*))$$

Úkoly na příští týden

- Seznámit se s přednáškovými poznámkami a souvisejícími materiály
- V souladu s rezervační tabulkou průběžně připravovat a odevzdávat individuální prezentace